IA para Desarrolladores

Sesión 2 · Explorar — los agentes en acción y el ciclo AI SDLC

CURSO · 20 HORAS · 10 SESIONES

Agenda de hoy

10'	10'	45'	15'	30'	10'
Repaso y lo que nos pidieron	Demo A: dos límites	Demo B: dos agentes construyen una app mientras vemos el ciclo AI SDLC	Demo: el repositorio del equipo y /intencion	Práctica: cada uno escribe la intención de su reto	Tarea y cierre

Hoy practican en su máquina: traigan Claude Code o Copilot listos y el repositorio del curso.

Lo que nos dejó la sesión 1

1

Un LLM predice tokens

No sabe, completa. Por eso inventa con seguridad cuando le falta contexto.

2

Cuatro formas de uso

Copilot y Claude Code, cada uno en VS Code y en la terminal: el mismo agente con otra interfaz.

3

El contexto manda

Lo que el asistente lee (archivo abierto, reglas, modelo) decide lo que responde.

Hoy: los agentes construyen de verdad, y aprendemos el método para dirigirlos.

Lo que nos pidieron y dónde lo veremos

Lo que pidieron	Sesión	Qué verán
Gastar menos tokens	S3 y en cada tarea	Qué llena la ventana, /context, /compact y /clear, skills y subagentes, modelo y esfuerzo según la tarea
Buenos prompts	S3	Objetivo, contexto, restricciones y criterio de terminado; cuándo un prompt se vuelve skill
Automatizar pruebas	S6 y S8	Prueba primero, datos sintéticos, hooks que corren pruebas y GitHub Actions en cada pull request
Corregir vulnerabilidades	S7	Revisión de seguridad con el agente, análisis de dependencias y de código, corrección guiada
Flujo DevOps con GitHub	S5 y S8	Ramas, pull requests, revisión de Copilot y del agente, Actions y protección de rama

Todo esto termina dentro del agente del equipo: reglas, skills, agentes y hooks que lo dejan automatizado.

Los agentes en acción

Dos límites y una app construida a partir de requisitos

Demo A: dos límites, en las dos herramientas

1 Alucinación

```
¿Cómo uso
pandas.DataFrame.to_bank_ledger() para
exportar un libro contable?
```

Esa función no existe. Misma pregunta en Copilot Chat y en Claude Code: ¿alguno la inventa?

2 Datos sensibles

```
API_KEY = "sk-demo-…"
Archivo abierto en VS Code con los dos
paneles
```

Los dos ven el archivo abierto, aunque no se lo adjunten. ¿Qué acaba de salir del equipo?

Sus modelos: qué usar para qué

Claude Code

21 personas

sonnet: el día a día: entender, escribir, probar

opus: planear, depurar, refactors grandes

haiku: tareas rápidas y agentes verificadores

/model y /effort para cambiar

GitHub Copilot Business

3 personas

Claude Sonnet 5: el trabajo principal del curso

Claude Haiku 4.5: tareas rápidas

GPT-5.3-Codex: alternativa para código

Grok 4.6: disponible; para comparar

La lista depende de la licencia y de lo que habilite la organización: puede cambiar de una persona a otra.

Demo B: una app construida desde requisitos

Pregunta de negocio

Un cliente quiere un crédito de libre inversión. ¿Cuánto pagará cada mes, cuánto le costará en total y le alcanza con sus ingresos?

REQUISITOS.md: validaciones, cuota fija, tabla de amortización, capacidad de pago y una app web con Streamlit.
AGENTS.md: Decimal, tasas E.A., dominio separado de la interfaz.

`simulador-copilot`
Copilot Chat · Agent

`simulador-claude-code`
Claude Code CLI

```
/plan Lee REQUISITOS.md y AGENTS.md y construye
la aplicación que describen.
→ revisamos los dos planes → aprobamos
```

Primero el plan; lo aprobamos y construyen mientras vemos el ciclo.

El ciclo AI SDLC

Cómo trabajaremos con agentes hasta el proyecto final

Primero: ¿qué es el SDLC?

El ciclo de vida del desarrollo de software (SDLC) son las etapas por las que pasa todo cambio, desde la necesidad hasta producción. Cada etapa tiene un entregable y un responsable.

1. **Requisitos** → Qué se necesita y para quién
2. **Diseño** → Cómo se va a resolver
3. **Construcción** → El código
4. **Pruebas** → ¿Hace lo que debe?
5. **Despliegue** → Pasa a producción con aprobación
6. **Mantenimiento** → Errores, cambios, mejoras

En un banco, el SDLC además deja rastro: quién pidió el cambio, quién lo aprobó, qué se probó y cuándo pasó a producción. Eso no cambia con la IA; lo que cambia es quién hace el trabajo de cada etapa.

Tres formas de usar IA en el SDLC

1

Asistida

La persona hace el trabajo y la IA ayuda: autocompletar, preguntas en el chat.

Lo que la mayoría hace hoy

2

Con agentes

El agente ejecuta tareas de varios pasos: lee, edita, prueba. La persona revisa el resultado.

Lo que vieron en la demo B

3

Nativa de IA (AI SDLC)

Se rediseña el proceso: agentes en cada etapa, cada etapa deja un artefacto y una persona lo aprueba.

A donde llega el curso

Escribir código ya no es el cuello de botella

1

Antes

Construir era lo lento. Planear, revisar y aprobar se hacían al ritmo del código.

2

Ahora

Un agente escribe código en minutos. Lo lento es lo humano: decidir qué construir, revisar y aprobar.

3

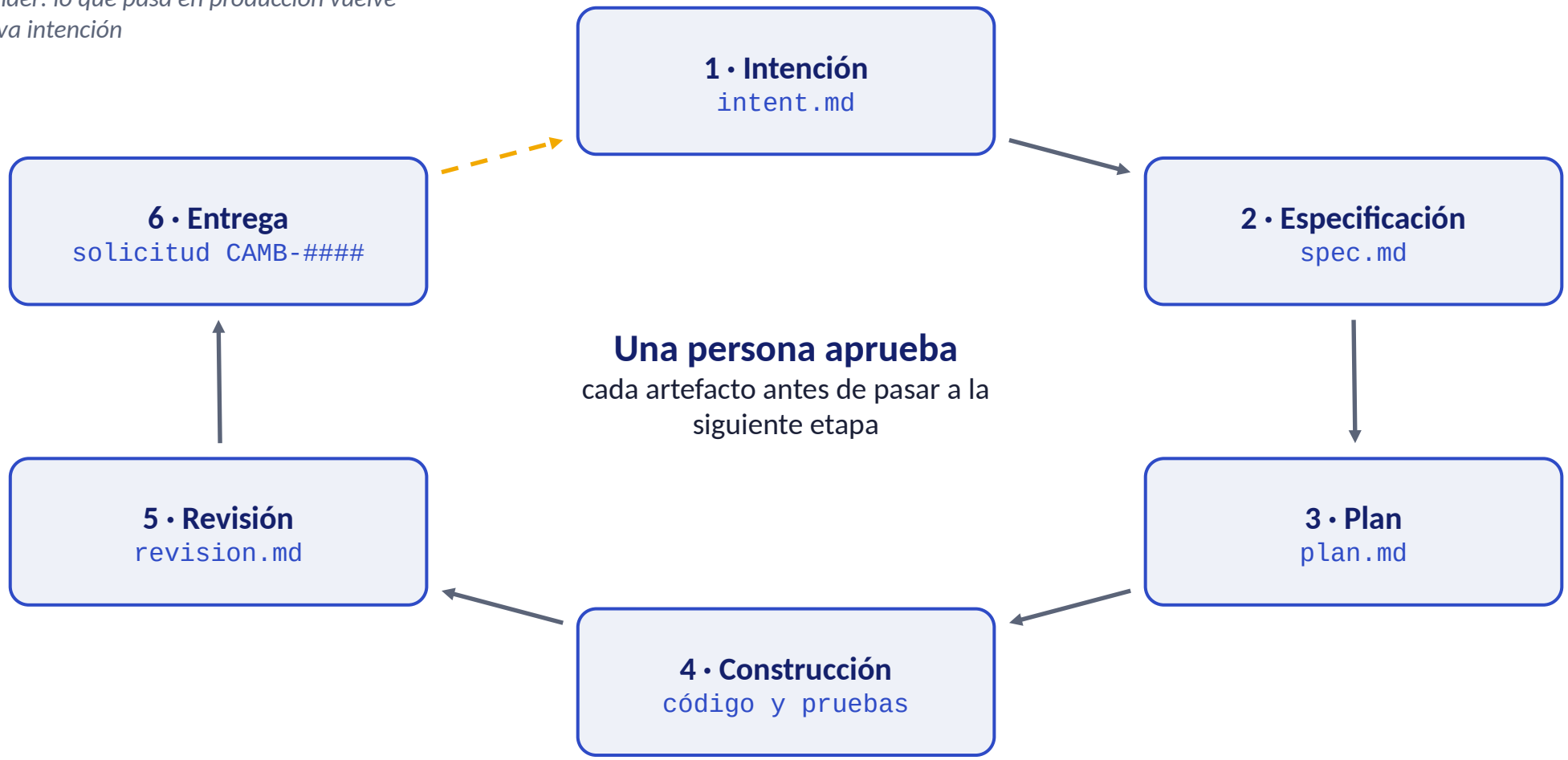
Entonces

Rediseñamos cada etapa: el agente redacta y construye, y la persona decide con un artefacto escrito delante.

"Build is no longer the constraint — the human-speed steps around it are." The AI-native SDLC playbook, Anthropic

El ciclo AI SDLC: seis etapas en un bucle

Operar y aprender: lo que pasa en producción vuelve como una nueva intención



Qué hace el agente y qué decide la persona

Etapa	El agente	La persona	Pieza del agente	Llega en
1 · Intención	Pregunta lo que falta y redacta intent.md	El dueño del producto corrige y aprueba	skill /intencion	Hoy
2 · Especificación	Redacta spec.md con las políticas del banco	El dueño del producto aprueba	skill /spec + MCP	S3
3 · Plan	Propone los pasos sin tocar código	El líder técnico aprueba plan.md	agente Arquitecto	S4
4 · Construcción	Implementa y corre las pruebas	El desarrollador revisa el diff	agente Desarrollador + hooks	S4–S6
5 · Revisión	Revisa, verifica y reporta hallazgos	El dueño del código aprueba el pull request	agentes Revisor y Verificador	S5–S7
6 · Entrega	Prepara la solicitud de cambio	El gestor de cambios autoriza	crear_solicitud_cambio + hook	S8

El agente nunca aprueba su propio trabajo ni pasa a producción.

Cómo se ve en el repositorio: la trazabilidad sale sola

```
docs/sdlc/validador-transferencias/
├── intent.md      Estado: aprobado
├── spec.md        Estado: aprobado
├── plan.md        Estado: aprobado
└── revision.md    Estado: borrador

$ git log --oneline
9c1e2aa Revisión del validador (borrador)
71b0d3f Código y pruebas del validador
4d2f9e1 Plan aprobado por líder técnico
2a8c7b0 Spec aprobada por dueño del producto
1f3e6d2 Intención aprobada
```

Cada artefacto es un archivo Markdown versionado: quién, qué y cuándo.

Aprobar es cambiar el Estado y hacer commit, o aprobar el pull request.

Si cambia el requisito, se actualiza intent.md o spec.md y el agente rehace desde ahí.

Sirve para auditoría sin documentos aparte.

¿Qué es una skill?

Una carpeta con instrucciones y archivos de apoyo que le enseñan al agente a hacer una tarea concreta, como la hace el equipo.

Se carga solo cuando hace falta: el agente ve el nombre y la descripción; lee el resto cuando la tarea lo pide. Gasta menos tokens que poner todo en AGENTS.md.

Dos formas de usarla: ustedes la llaman con /nombre, o el agente la elige solo porque su descripción encaja con la tarea.

Puede traer más: plantillas, ejemplos y scripts en la misma carpeta.

Mismo archivo en las dos: SKILL.md en .claude/skills/ (Claude Code) y .github/skills/ (Copilot).

Skills del curso

/intencion · hoy

/spec · S3

/plan · S4

/revisar · S5

/pruebas · S6

revision-seguridad · S7

Regla práctica: un prompt que repiten mucho se vuelve skill.

¿Qué es un hook?

Un comando que la herramienta ejecuta automáticamente en un momento fijo del trabajo del agente. No depende de que el modelo obedezca.

Inicio de sesión

Cargar contexto: rama, pendientes

→

Antes de usar una herramienta

Puede bloquear: leer .env, borrar archivos, "desplegar" sin aprobación

→

Después de usar una herramienta

Revisar: correr linter y pruebas después de editar

→

Al terminar

Verificar: ¿quedaron pruebas?, ¿pasan?

	Claude Code	GitHub Copilot
Dónde	.claude/settings.json	.github/hooks/*.json
Momentos	SessionStart, PreToolUse, PostToolUse, Stop…	sessionStart, preToolUse, postToolUse, agentStop…

Todo gira alrededor del agente

La herramienta: Claude Code o Copilot

Hooks y permisos los ejecuta la herramienta antes y después de cada acción del agente; el agente no los elige ni los puede saltar

Lo que lee

Instrucciones: AGENTS.md, CLAUDE.md, siempre

Skills: solo cuando la tarea lo pide o usted escribe /nombre

El agente

Un modelo + un ciclo: lee, planea, actúa, verifica, y repite

Decide qué leer, qué herramienta usar y a quién delegar

Lo que usa

Herramientas propias: leer, editar, terminal

Herramientas MCP: consultar_politica, catálogo…

A quién delega

Subagentes por rol: Revisor, Verificador

Quién activa cada pieza y cuál se cumple siempre

Pieza	Qué es	¿Quién la activa?	¿Se cumple siempre?	Ejemplo
Instrucciones	Reglas que siempre aplican	La herramienta las carga al empezar	No: el modelo puede ignorarlas	Dinero en Decimal
Skill	Cómo hacer una tarea	El agente, o usted con /nombre	No: es un consejo	/intencion
Herramienta MCP	Una acción en otro sistema	El agente decide llamarla	La regla la pone ese sistema	consultar_politica
Subagente	Un rol con sus herramientas y su modelo	El agente le delega	Sus permisos sí	Revisor de solo lectura
Hook	Un comando automático	La herramienta, no el agente	Sí	Bloquear .env

Una skill aconseja, un hook obliga.

Dos principios que no se negocian

Los agentes proponen, las personas aprueban

Ningún agente aprueba su propio trabajo ni entrega a producción. Cada artefacto pasa de borrador a aprobado con el nombre de una persona.

Una skill aconseja, un hook obliga

Lo que el agente debería hacer va en reglas y skills. Lo que nunca puede pasar (datos reales, secretos, entregar sin aprobación) va en un hook que lo bloquea.

Las piezas del agente en las dos herramientas

Pieza	Claude Code	GitHub Copilot	Sesión
Reglas del equipo	CLAUDE.md → AGENTS.md	AGENTS.md, copilot-instructions.md	S2
Skills (/intencion, /spec…)	.claude/skills/	.github/skills/ (mismo archivo)	S2–S7
Reglas por tipo de archivo	.claude/rules/	.github/instructions/	S3
Herramientas del banco (MCP)	.mcp.json	.vscode/mcp.json	S3–S8
Agentes por rol	.claude/agents/	.github/agents/	S4–S6
Hooks (puntos de control)	.claude/settings.json	.github/hooks/	S6–S8

Se escriben una vez y se usan en las dos. Al final, todo se empaqueta en la plantilla del proyecto.

Dos carpetas, el mismo agente: .claude y .github

Claude Code

```
mi-reto/
├── CLAUDE.md          entrada; importa AGENTS.md
├── .mcp.json          herramientas MCP    · S3
└── .claude/
    ├── skills/
    │   └── intencion/  la skill           · hoy
    ├── rules/          por tipo de archivo · S3
    ├── agents/         agentes por rol     · S4
    └── settings.json   permisos y hooks    · S6
```

GitHub Copilot

```
mi-reto/
├── AGENTS.md          reglas (las leen los dos)
├── .vscode/mcp.json   herramientas MCP    · S3
└── .github/
    ├── copilot-instructions.md   entrada
    ├── skills/
    │   └── intencion/  copia de la skill · hoy
    ├── instructions/   por tipo de archivo · S3
    ├── agents/         agentes por rol     · S4
    └── hooks/          puntos de control   · S6
```

En la raíz, lo común: AGENTS.md, banco.yaml y docs/sdlc. Cada herramienta busca lo suyo en su carpeta; herramientas/sincronizar_skills.py copia .claude/skills a .github/skills.

¿Y .vscode/settings.json?

```
// mi-reto/.vscode/settings.json
{
  "chat.useAgentsMdFile": true,
  "python.terminal.activateEnvironment": true
}
```

Qué es: la configuración de VS Code solo para esta carpeta. Gana sobre la configuración personal y viaja con el repositorio: todo el equipo trabaja igual.

chat.useAgentsMdFile: Copilot Chat lee AGENTS.md como instrucciones.

python.terminal.activateEnvironment: la terminal abre con el entorno de Python activo.

No confundir

.vscode/settings.json · del editor: cómo se comporta VS Code (y Copilot Chat dentro de él).

.claude/settings.json · de Claude Code: permisos y hooks. Llega en la sesión 6.

.vscode/mcp.json · las herramientas MCP para Copilot. Llega en la sesión 3.

Banco Orquídea: un banco ficticio en un archivo

```
banco:
  nombre: Banco Orquídea
  ficticio: true
productos:
  credito_consumo:
    tasa_maxima_ea: 0.26
    endeudamiento_maximo: 0.40
  transferencias:
    limite_diario: 10000000
datos:
  clasificacion: [publica, interna,
    confidencial, restringida]
convenciones:
  dinero: "Decimal, nunca float"
```

Todo es inventado: productos, límites, políticas y herramientas.

Un solo archivo: banco.yaml. Las reglas, las skills y las herramientas lo leen.

Después del curso, el banco reemplaza banco.yaml y AGENTS.md con sus propias definiciones.

Nunca valores inventados: si una regla no está en banco.yaml, el agente pregunta.

¿Qué construyeron? ¿Y era lo correcto?

1

Sus pruebas

python -m pytest -q en cada carpeta: ¿pasan?

2

Pruebas de aceptación

26 pruebas que el agente nunca vio: ¿cumple los requisitos?

3

La app

streamlit run app.py: $10 M, 12 meses, 24 % E.A. → cuota $934.525

4

La pregunta

¿Qué decidió cada agente que no estaba en los requisitos?

Construir fue rápido porque alguien ya había decidido qué construir. En su trabajo, esa decisión no existe todavía: el ciclo empieza por la intención.

Demo: el repositorio del equipo

La primera pieza del agente: la skill /intencion

Qué trae el repositorio del equipo

```
mi-reto/
├── banco.yaml
├── AGENTS.md
├── CLAUDE.md
├── .github/
│   ├── copilot-instructions.md
│   └── skills/intencion/
├── .claude/
│   └── skills/intencion/
├── docs/sdlc/
└── herramientas/
    ├── verificar_equipo.py
    └── sincronizar_skills.py
```

banco.yaml • las definiciones del banco ficticio

AGENTS.md • reglas del equipo y el ciclo; lo lee Copilot

CLAUDE.md • importa AGENTS.md y banco.yaml; lo lee Claude Code

skills/intencion • la primera skill, en las dos carpetas

docs/sdlc • un artefacto por etapa y por caso

herramientas • verificar el repositorio y copiar skills

Anatomía de una skill: el mismo archivo para las dos

```
---
name: intencion
description: Convierte una idea o necesidad
  de negocio en docs/sdlc/<caso>/intent.md.
  Úsela cuando alguien proponga un caso
  de uso nuevo, antes de cualquier código.
---

# Redactar la intención de un caso de uso
1. Lea banco.yaml y AGENTS.md
2. Pregunte lo que falte (máximo 5)
3. Escriba intent.md con la plantilla
4. Deje el estado en borrador
```

name

Cómo se invoca: /intencion

description

Lo que el agente lee para decidir cuándo usarla. Es la parte más importante.

Pasos y reglas

Lo que hace, en orden, y lo que nunca debe hacer

Archivos de apoyo

plantilla-intent.md vive en la misma carpeta

Demo: /intencion con el simulador de crédito

```
/intencion Los asesores calculan los créditos de consumo en hojas de cálculo distintas
en cada oficina y los resultados no coinciden. Queremos un simulador único que diga
cuánto paga el cliente cada mes, cuánto le cuesta en total y si le alcanza.
```

1

Pregunta

Hasta 5 preguntas sobre problema, usuarios, métrica, datos y alcance

2

Escribe

docs/sdlc/simulador-credito/intent.md con las 11 secciones

3

Consulta el banco

Clasifica los datos y cita reglas con su clave de banco.yaml

4

Lo deja en borrador

No se aprueba a sí mismo: lo aprueba el dueño del producto

En Copilot Chat es igual: agente Agent, modo Interactive, escriban / y aparece intencion.

¿Cuándo se aprueba un intent.md?

- Están las 11 secciones; ninguna vacía (o dice "No aplica" y por qué)
- El problema está en lenguaje de negocio, sin soluciones técnicas
- La métrica de éxito se puede medir: hoy y meta
- Los datos están clasificados con banco.yaml y son sintéticos
- Las reglas citan su clave de banco.yaml; no hay valores inventados
- Lo que queda fuera está escrito en "No incluye"

Aprobar = dos cambios y un commit

```
Estado: aprobado
Aprobado por: <nombre>

git commit -am
  "Intención aprobada"
```

Retos para elegir

Reto sugerido	Tipo de entregable
Validador de transferencias	Componente o API
Modernizar la conciliación heredada	Código heredado
Alertas de transacciones inusuales	Proceso ETL + reglas
Generador de extractos	Componente + reporte
API de productos de un cliente	API
Calculadora de capacidad de pago	Componente
Pruebas para un módulo existente	Suite de pruebas

Por ahora, individual: cada uno en su máquina, con un repositorio local

Sin GitHub todavía: el trabajo compartido llega en la sesión 5

Datos sintéticos siempre

Pueden proponer un reto propio del trabajo diario

Cada reto trae su idea para /intencion en RETOS_SUGERIDOS.md

El simulador es el ejemplo resuelto: no es un reto

Manos a la obra: la intención de mi reto

5'

Crear mi repositorio

Desde la carpeta del curso: crear_repositorio.py ../mi-reto --proyecto plantilla-equipo y verificar_equipo.py

15'

Escribir la intención

Abrir mi-reto en VS Code. /intencion con la idea del reto; responder sus preguntas

5'

Revisar juntos

Una o dos personas comparten pantalla; los demás revisan el suyo con la lista de aprobación

5'

Aprobar y registrar

Editar Estado y Aprobado por; git add y git commit

¿Algo no funciona? verificar_equipo.py dice qué falta. Si /intencion no aparece, abra VS Code en la carpeta mi-reto.

Para la sesión 3

1

Mi intent.md

Terminarlo: cerrar preguntas abiertas, aprobar y hacer commit. Enviarlo por el canal.

2

Instalar con tiempo

pip install pyyaml mcp · para el servidor de herramientas del banco.

3

Leer

Claude Code 101: instalación, primer prompt y flujo diario. GH-600: cierre del módulo 1.

4

Lo que viene

Reglas por tipo de archivo, la skill /spec y las herramientas del banco por MCP.

Escribir código es barato; decidir qué construir y aprobarlo es el trabajo.